

Operation — Cloud Token Blackout

CI/CD cloud-credential abuse · Intel ref RIU-INT-2026-1006-CTB · TLP:AMBER

Eswar Mahalingam

CTI / RIU · Candidate 13

GS-STU-DSOU-2026-039A

Identity

not the app

AdminAccess

via stolen key

PII + secrets

exfiltrated

7 days

log retention

16/16

tests pass

OIDC

the fix

CTI Lead's question — compromise the app, or the identity that builds and deploys it? **The identity.** A long-lived AWS key on a self-hosted runner was used from a hosting IP, escalated to AdministratorAccess, read PII + DB-backup buckets, pulled prod/db/master, and made a bucket public — valid credentials, zero app exploit. Rebuilding the runner does not contain it.

Mission 01 — Cloud Credential Exposure Card

Theme	Intel
How CI runners expose cloud creds	Secrets in env vars <code>~/.aws/credentials</code> on the runner; secrets echoed to build logs; a tampered workflow step (<code>env \ curl ...</code>); artifacts that embed tokens; a shared/self-hosted runner reused across repos
OIDC short-lived vs long-lived keys	OIDC (<code>AssumeRoleWithWebIdentity</code>) mints a credential scoped to the job that expires in minutes and is bound to the repo/branch claim. Long-lived access keys (<code>AKIA...</code>) never expire until rotated, carry the user's full policy, and work from anywhere — one leak = standing access
Why runners are attractive	They legitimately hold deploy-grade cloud permissions, run arbitrary build code, and sit between source and production — compromise one identity and you can reach code, cloud and secrets
Deployment creds → cloud persistence	With IAM write, an attacker creates a second access key, attaches admin, adds trust-policy backdoors, or makes a bucket public — persistence that survives rebuilding the runner
Impact of excessive IAM	A runner with <code>:*</code> or <code>AdministratorAccess</code> turns one key leak into full-account takeover; least-privilege would have capped blast radius to the artifact bucket
Why missing history hurts	CloudTrail retained 7 days → the first access, the OIDC-vs-key history, and the full blast radius may already be unrecoverable; investigators can't prove when access began

→ One-page card rendered in the PDF (`GraySentinel_RIU_CloudTokenBlackout_Report.pdf`).

Mission 02 — Threat actor & TTP mapping

Chain stage	Technique	ID	Evidence required	Expected telemetry	Confidence
Compromised runner	Valid Accounts: Cloud	T1078.004	tampered workflow, stale actor	GitHub audit: <code>workflow.edited</code> , <code>workflow_run</code>	High
	CI/CD poisoning	T1554 / supply-chain	new step leaking env	repo diff, run logs	Medium
Credential access	Unsecured Credentials	T1552.001	key in env/logs	<code>secret.retrieved</code> , build logs	High
	Cloud secrets	T1555.006	<code>GetSecretValue</code>	CloudTrail Secrets Manager	High
Cloud authentication	Valid Accounts: Cloud	T1078.004	key used off-runner	CloudTrail <code>GetCallerIdentity</code> , new sourceIP/ASN, no MFA	High
Privilege escalation	Account Manipulation: Additional Cloud Creds	T1098.001	<code>CreateAccessKey</code> , <code>AttachUserPolicy</code>	CloudTrail IAM	High
Data access	Data from Cloud Storage	T1530	<code>GetObject</code> on new buckets	CloudTrail S3 data events	High
Exfiltration	Transfer to Cloud Account / public ACL	T1537	<code>PutBucketPolicy</code> Principal:*	CloudTrail	High

Resembles: Initial Access (stale identity) → **Credential Theft** (runner key) → Cloud Persistence (new key + admin + public bucket) → **Data Theft** (PII/backups/secrets) → Supply-Chain Abuse (deploy-capable identity).

Mission 03 — OSINT & exposure methodology (OSINT only — no unauthorized access)

Surface	What a defender searches	VALID IoC vs FALSE POSITIVE
GitHub	code search for <code>AKIA</code> , <code>aws_secret_access_key</code> , <code>.env</code> , <code>id_rsa</code> ; workflow files referencing plaintext secrets; public forks of internal repos	IoC: live-format key tied to your account ID / a real resource. FP: <code>AKIAIOSFODNN7EXAMPLE</code> (AWS docs sample), rotated/placeholder keys
Google dorks	<code>site:github.com "AWS_SECRET"</code> , <code>filetype:yml "aws-access-key-id"</code> , <code>intitle:"index of" ".env"</code> , exposed <code>*.tfstate</code>	IoC: your org's domain/resources present. FP: tutorials, honeytokens, another org's data
Cloud exposure	public S3 / Azure Blob listing; IAM policies with <code>*:*</code> ; public CI artifacts; <code>*.s3.amazonaws.com</code> 200s	IoC: your bucket resolves + lists objects. FP: intentionally public assets (static site, downloads)
Linux runner forensics	<code>~/ .bash_history</code> , <code>env</code> , <code>/home/runner/work/*</code> , <code>~/ .aws/</code> , <code>/var/log/auth.log</code> , recently-modified <code>credentials/config</code>	IoC: key use + outbound to an unknown host right after a build. FP: the CI tool's own cache/legit deploy calls

Rule: a finding is a **valid IoC only when it ties to your account ID, resource ARN, or domain**; sample keys, honeytokens and third-party data are false positives.

Mission 04 — Indicator & intel report

IoC list (from the lab analyzer): - Suspicious principal: IAM user `runner-deploy` (should only ever be the OIDC role `github-ci-deploy`) - Source IP: `203.0.113.77` (hosting ASN, first-seen) - Access key abused: `AKIAEXAMPLE7RUNNER01`; **new key created:** `AKIAEXAMPLEN3WKEY99` - IAM change: `AttachUserPolicy` → `AdministratorAccess` - Object-storage: `GetObject` on `example-prod-pii`, `example-db-backups` (never accessed by this principal before) - Public bucket: `example-prod-pii` set to `Principal:*` - CI runner: `deploy.yml` edited to leak `env`; actor `ex.contractor` (terminated, still in automation)

Detection hypotheses: Sigma pack `cicd_cloud_token_abuse.sigma.yml` — (1) long-lived key off CI ranges, (2) IAM privesc, (3) secret retrieval from unusual source, (4) bucket made public; correlation fires CRITICAL when ≥2 hit for one principal in 10 min.

Diamond Model

Vertex	Value
Adversary	Access broker / intrusion actor advertising "developer infrastructure" on dark-web chatter
Capability	Stolen long-lived CI token, IAM privesc, S3/Secrets collection, public-ACL exfil (LOTL — no malware)
Infrastructure	Hosting IP <code>203.0.113.77</code> ; exfil host <code>cdn-sync.example.net</code> ; the victim's own CI runner
Victim	Tech/fintech/SaaS running GitHub CI/CD into AWS/Azure with long-lived runner creds + 7-day logs

Kill chain: Runner Compromise (stale identity + tampered workflow) → Credential Access (env key) → Cloud Access (`GetCallerIdentity` off-runner) → Privilege Escalation (`CreateAccessKey` + `AttachUserPolicy`) → Collection (PII/backups/secrets) → Exfiltration (public bucket / POST to C2).

Senior question — rebuild the runner today, why still HIGH? Rebuilding the box removes the *host*, not the *access*.

Stolen credentials (the original key + the attacker-created `AKIAEXAMPLEN3WKEY99`) still authenticate from anywhere.

Active cloud sessions (assumed-role STS tokens) keep working until expiry. **IAM persistence** (the admin attachment, trust-policy or public-bucket changes) survives. **Data access** already happened — PII, DB backups and `prod/db/master` are out, so secrets must be assumed burned. HIGH until every key is revoked, IAM diffed to a known-good baseline, the DB secret rotated, and the public bucket closed.

Mission 05 — CISO intel brief (1 page — in the PDF)

TL;DR: A CI/CD identity, not the app, was compromised. A long-lived AWS key on a self-hosted runner was used from an external host to gain admin, read customer-PII and backup buckets, pull the production DB secret, and expose a bucket publicly. Rebuilding the runner does not contain it.

Who is at risk: any team with self-hosted runners holding long-lived cloud keys and short log retention. **What's abused:** the deploy identity's standing permissions.

Business impact: cloud account takeover · customer-data exposure (PII + backups) · CI/CD compromise · software supply-chain risk (deploy-capable identity can ship code) · production deployment manipulation.

SOC actions — Next 24 h: revoke `AKIAEXAMPLE7RUNNER01` + the attacker-created key; review all cloud auth from non-CI sources; hunt CI runners for tampered workflows/env leaks; preserve CloudTrail + GitHub audit now (7-day window); review IAM changes (`CreateAccessKey`/`AttachUserPolicy`) and close the public bucket. **Next 7 days:** rotate all long-lived secrets + the DB master secret; review IAM for `*:*/admin` on runners; enable CloudTrail data events +

GuardDuty; audit CI/CD trust relationships and remove stale identities (ex. [contractor](#)). **Next 30 days:** move to short-lived OIDC workload identity (kill long-lived keys); enforce least privilege; centralize + extend cloud logging (≥ 90 days); build CI/CD threat-hunting playbooks; require signed/provenance-checked artifacts.

Senior intel question — A vs B

B is categorically worse. (A) one stolen credential + one bucket is a bounded data-exposure incident. (B) a compromised **CI/CD identity that can deploy code, create cloud resources, read secrets and change production** is the keys to the factory.

- **Data value:** A = one bucket; B = everything the pipeline can reach, plus the ability to plant backdoors in *future* releases that ship to customers.
- **Persistence:** A ends when the key is revoked; B lets the attacker mint new credentials, add IAM backdoors, and re-enter through the trusted pipeline — revoking one key doesn't help.
- **Privilege:** A is scoped to one credential; B holds deploy + IAM + secrets authority.
- **Attacker ROI:** A is a smash-and-grab; B is a supply-chain foothold they can monetize repeatedly (resale as "developer infrastructure", ransomware staging, downstream customer compromise). Highest ROI, hardest to evict.

Identity is the new perimeter — if the CI/CD identity falls, production follows.

MITRE ATT&CK map

T1078.004 Valid Accounts: Cloud · T1552.001 Credentials in Files · T1555.006 Cloud Secrets · T1098.001 Additional Cloud Credentials · T1530 Data from Cloud Storage · T1537 Transfer to Cloud Account · T1554 Compromise Host Software Binary (CI poisoning) · T1070 (short retention aids evasion)

References: MITRE ATT&CK Cloud Account / Valid Accounts; AWS CloudTrail docs; Microsoft Entra audit/sign-in logs; GitHub Actions security docs; CISA cloud guidance. All data synthetic; no unauthorized access; no exploit code.